Part I — Speech — recognition & synthesis

ml-ladder writeup · part I

2026-10-08

Contents

I1 — Audio front end from scratch **2**
The idea . . . 2
The math . . . 3
What we measure, and why . . . 4
What I built . . . 4
Results . . . 4
What surprised me . . . 7
Open threads . . . 7
Further reading . . . 7

I2 — CTC speech recognition, and Whisper on the same audio **7**
The idea . . . 8
The math . . . 8
What we measure, and why . . . 9
What I built . . . 9
Results . . . 9
What surprised me . . . 11
Open threads . . . 11
Further reading . . . 12

I3 — Text-to-speech, evaluated without listening **12**
The idea . . . 12
The math . . . 12
What we measure, and why . . . 13
What I built . . . 13
Results . . . 13
What surprised me . . . 14
Open threads . . . 16
Further reading . . . 16

I4 — Streaming and latency **16**
The idea . . . 16
The math . . . 16
What we measure, and why . . . 17
What I built . . . 17
Results . . . 17
What surprised me . . . 18
Open threads . . . 19

Further reading 19

Whisper through the candle fork, actually run **19**
The example: empty transcriptions, and why 19
The executor: it transcribes, and it has four defects of its own 21
The executor's defects, fixed and re-measured 23
Open threads 23

What part I settled **23**

Status: complete (I1–I4) · **Constraint:** 8 kHz spoken digits from six speakers; four cores and no GPU, so widths and budgets are small and stated per rung
Numbers: seeds 0–2, reference seed 0 · raw values in `data/i-metrics.json`; the candle-fork run in `data/i-real-models.json`
Reproduce: `ladder run I1` … `I4` (needs `uv sync --extra torch --extra speech`), or `uv run python tools/writeup_figures.py I`
Hardware: all four rungs on 4 CPU cores; Whisper through the candle fork's example and through ignite-catalog's executor on a Kaggle Tesla T4

the third modality; reuses C's sequence models and H's CTC on a signal that is continuous in time

Parts B through H worked on grids and sequences of symbols. Speech is neither: it is a pressure wave, and every decision about how to turn it into something a network can read happens *before* the network exists. That makes the front end the subject rather than the plumbing, which is why I1 comes first and the other three rungs all use its output unchanged.

The four rungs are four different ways of asking **what a metric on speech is actually measuring**, because in this part more than any other the obvious number keeps being about something else:

- **I1** — two implementations of the same front end differ by 10^{-6} on three stages and by 3.5×10^{-2} on the fourth, and the cause is a convention nobody writes down.
- **I2** — the recogniser's error rate depends more on a normalisation choice I argued for in prose and got backwards than on anything architectural, and the *split* is worth 5× on its own.
- **I3** — synthesis has no objective score, so the rung reports three, and they disagree about the vocoder by two orders of magnitude.
- **I4** — accuracy and latency are the same quantity seen from two ends, and the exchange rate is measurable.

A note on the corpus. The Free Spoken Digit Dataset is 3,000 recordings of the digits 0–9 by six speakers at 8 kHz. It is small enough to run on a laptop and too small for any result here to transfer as a number. What transfers is the *shape*: the leak a shuffled split creates, the direction the normalisation choice moves, the fact that a vocoder can cost everything on one metric and nothing on another.

I1 — Audio front end from scratch

Question. The STFT/mel/MFCC stack is four decisions copied from the 1980s. What is each one doing, and would you notice if you got one wrong?

The idea

A microphone records **air pressure over time**: 8000 numbers per second for this corpus. Speech models almost never read that raw waveform. They read a **spectrogram**, a picture of which frequencies are present at each moment, because that is where the information that distinguishes sounds lives (vowels are patterns of resonant frequencies called *formants*).

Turning one into the other takes a fixed stack of steps, the **front end**:

1. **Framing** — cut the signal into short overlapping slices (32 ms, every 8 ms), short enough that speech is roughly steady within one.

2. **Windowing** — taper each slice to zero at its edges (a Hann window), so the frequency analysis does not see an artificial jump.
3. **FFT → power spectrum** — measure the energy at each frequency in each slice.
4. **Mel filterbank** — pool the frequencies into 40 bands spaced the way human hearing resolves pitch: fine at low frequencies, coarse at high ones.
5. **Log** — compress the huge dynamic range, as hearing does. The result, the **log-mel spectrogram**, is what I2–I4 and Whisper read.
6. **DCT → MFCCs** (optional) — summarise each log-mel column by its smooth envelope in 13 decorrelated numbers, the classical speech feature.

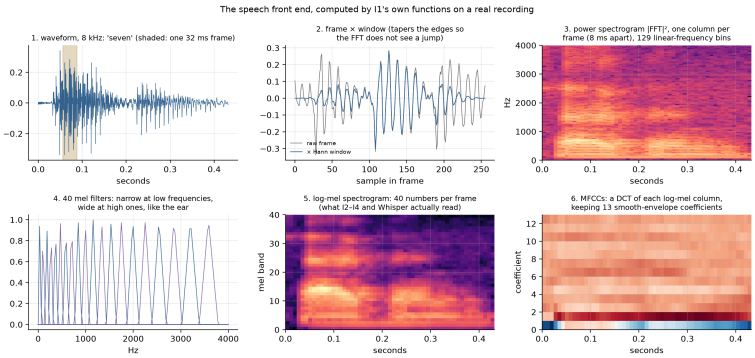


Figure 1: The front end, stage by stage

A real recording of "seven" through the rung's own functions. The waveform becomes a power spectrogram (formant bands visible as horizontal stripes), then 40 mel bands, then 13 MFCCs.

The math

Framing is a trade with a fixed exchange rate. A window of N samples at rate f_s gives frequency bins f_s/N apart, so two tones closer than about that cannot be told apart; the same window smears an instantaneous event across N samples. Sharpening either blurs the other, and the product is bounded below — the discrete cousin of the Gabor limit, and the reason a "better window" does not exist.

The mel scale is a claim about ears, not signals. Nothing in the physics distinguishes 500 Hz from 3 kHz; the claim is that human frequency discrimination is roughly logarithmic above ~700 Hz, encoded as

$$m = 2595 \log_{10}\left(1 + \frac{f}{700}\right)$$

with triangular filters evenly spaced in m. Two conventions are in use — this one (HTK) and Slaney's, which is linear below 1 kHz — and they put every filter edge in a different place, so "the mel scale" is not a well-defined phrase without saying which.

The log makes the source separable, and the DCT decorrelates. Speech is a glottal source filtered by a vocal tract, which is multiplication in the spectrum and therefore addition after a log — the entire cepstral idea. The DCT then exists for a specific and now-obsolete reason: adjacent mel filters overlap by construction, so their outputs are strongly correlated, and a diagonal-covariance Gaussian model could not cope with that. It is an orthonormal DCT-II, so truncating to 13 coefficients is a projection whose residual can be measured rather than an arbitrary crop.

What we measure, and why

- **Relative error against torchaudio, stage by stage** — each stage is checked separately against a reference implementation, so a mismatch is pinned to the stage that causes it, not discovered at the end of the pipeline.
- **Frequency resolution** — whether two tones a given distance apart show up as two peaks. **Time resolution** — how far a single click smears across frames. Together they measure the window-length trade-off the math derives.
- **Off-diagonal correlation of MFCCs vs log-mels** — the reason for the DCT step, measured: MFCC coefficients should be much less correlated with each other than neighbouring mel bands are.

What I built

Framing, windowing, the power spectrum, the mel warping, the filterbank, the log and the DCT, in numpy. Not the FFT — that is a numerical-methods exercise, not a speech one. Each stage is diffed against torchaudio *separately*, so a disagreement lands on the stage that caused it instead of on the end of the pipeline.

Results

Medians over seeds 0–2 (the front end is deterministic; only the clip sample varies).

stage	relative error vs torchaudio
filterbank	2.5×10^{-6}
power spectrogram	1.8×10^{-7}
mel spectrogram	1.1×10^{-6}
MFCC	1.3×10^{-6}
MFCC *with a clamped log instead of an offset*	3.5×10^{-2}

The time–frequency trade, measured rather than asserted:

window	resolves	click smeared over	product
4 ms	640 Hz	3 ms	1920
8 ms	320 Hz	6 ms	1920
16 ms	160 Hz	12 ms	1920
32 ms	80 Hz	24 ms	1920
64 ms	40 Hz	48 ms	1920
128 ms	20 Hz	96 ms	1920

Constant to **1.00×** across a 32× change in window length.

The mel warping puts **28 of 40** filters below 2 kHz — the lower half of the band — with centre spacing growing from 37.5 Hz to 187.5 Hz, a factor of **5.0**. The DCT drops mean off-diagonal correlation from **0.695 to 0.177**, a factor of **4.0**, and keeping 13 of 40 coefficients loses **10.9%** of the log-mel energy.

Each highlighted step throws information away on purpose

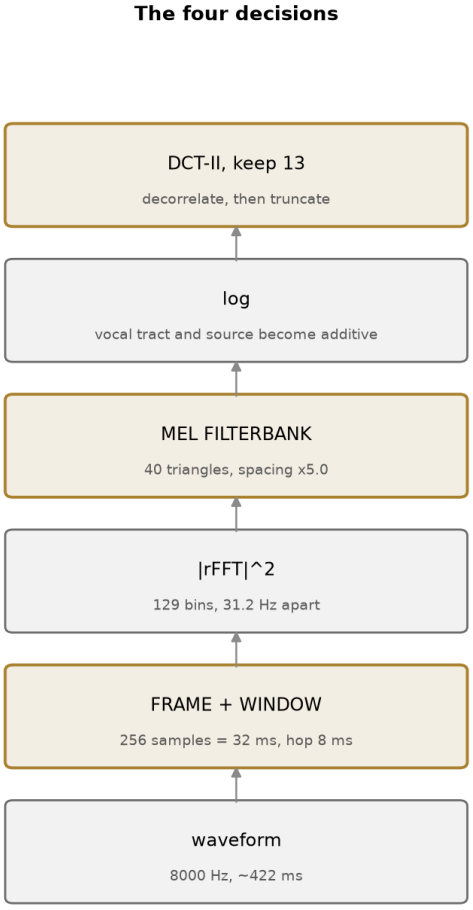


Figure 2: I1 pipeline

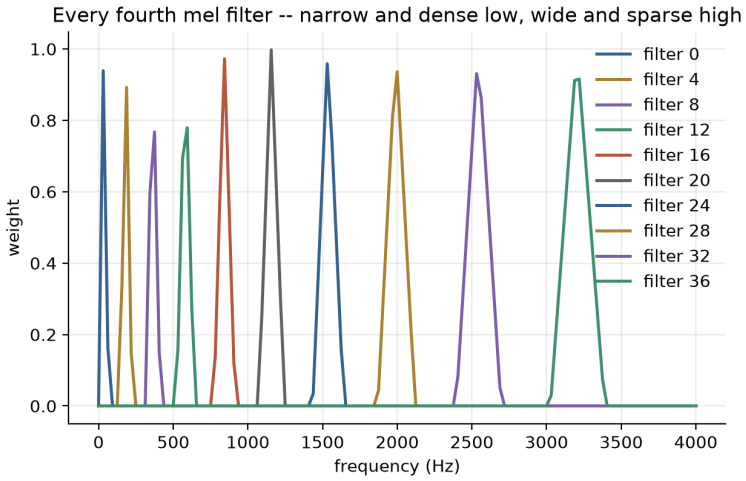


Figure 3: I1 filterbank

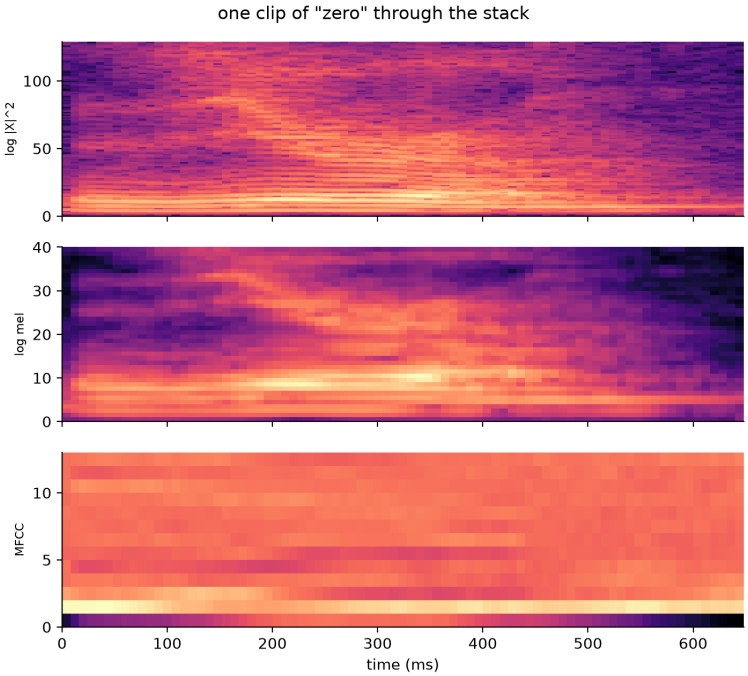


Figure 4: I1 stages

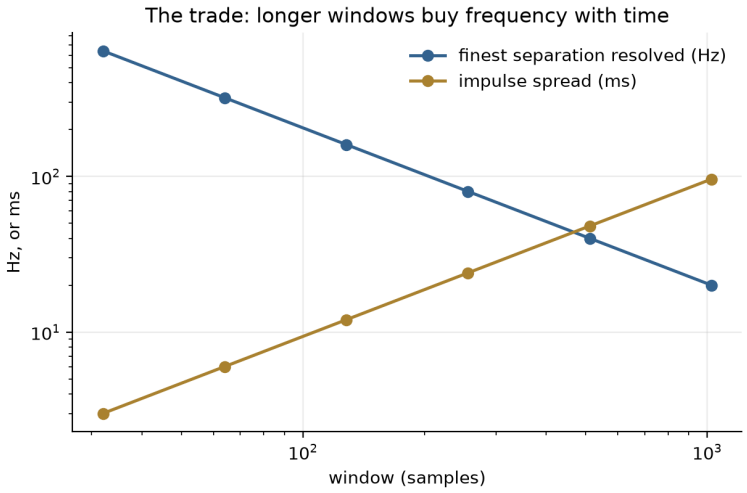


Figure 5: I1 resolution

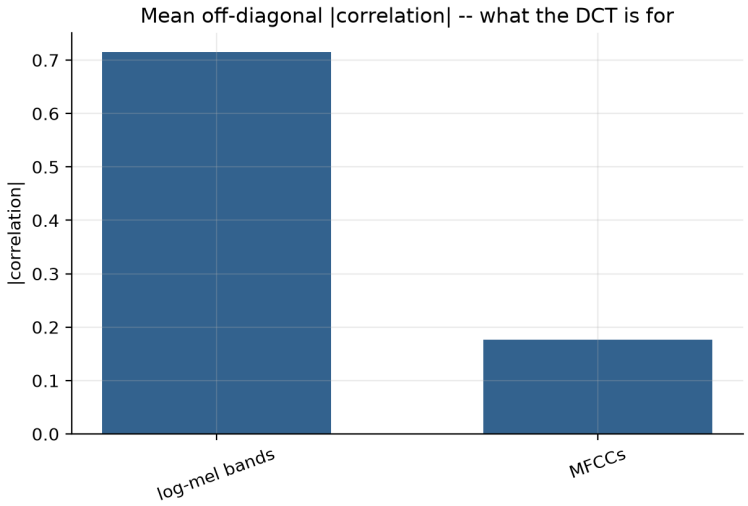


Figure 6: I1 decorrelation

What surprised me

The MFCCs disagreed by four orders of magnitude more than every other stage, and the cause is a line nobody documents. $\log 0$ is undefined, so every implementation picks a guard, and there are two: torchaudio's is an **offset**, $\log(x + 10^{-6})$; the obvious reading is a **clamp**, $\log(\max(x, 10^{-10}))$. They are not interchangeable. The quietest mel energy in this corpus is 9.5×10^{-9} — *below* torchaudio's offset — so the offset is not guarding anything, it is shifting every quiet frame, and choosing wrong costs **27,490×** the agreement. Three stages matching to 10^{-6} is what made the fourth legible; a single end-to-end diff would have shown one number and no way to locate it.

The product came out exactly 1920 Hz·ms at every window length, not approximately. I expected a trend with noise and got an identity, because the two quantities I measured are both counted in units of the same window. That is a good outcome for a claim that is usually stated as a vague trade-off.

Open threads

The Slaney mel convention is not implemented, so "which mel" is documented rather than measured. And the filterbank here is fixed; learned filterbanks (SincNet, LEAF) are the obvious comparison and would test whether 1980s perceptual modelling still earns its place.

Further reading

Davis & Mermelstein (1980) for MFCCs; Gabor (1946) for the limit the window sweep traces out.

I2 — CTC speech recognition, and Whisper on the same audio

Question. H3's CTC recogniser worked on image columns. Does it transfer to audio frames unchanged — and what does a pretrained model's error rate tell you that a small model's does not?

The idea

Automatic speech recognition (**ASR**) turns audio into text. This rung uses two very different systems on the same held-out speaker:

- **A small CTC recogniser**, trained here: H3's text-line reader moved from image columns to audio frames. A few convolutions over the frequency axis, a bidirectional GRU across time, and a letter-or-blank prediction per frame, collapsed by CTC into a word. Trained on four speakers saying digits.
- **Whisper**, OpenAI's pretrained model, used zero-shot. A transformer **encoder-decoder**: the encoder reads a log-mel spectrogram, and the decoder *writes* the transcript token by token like a language model, attending to the encoder (C2's attention, at scale). It was trained on 680,000 hours of transcribed audio, so it brings knowledge of language and of many voices that a model trained on 800 clips cannot have.

One property of Whisper matters later in this writeup. It always processes a **30-second window**, padding shorter audio with silence, so a half-second digit is a tiny fraction of what it sees.

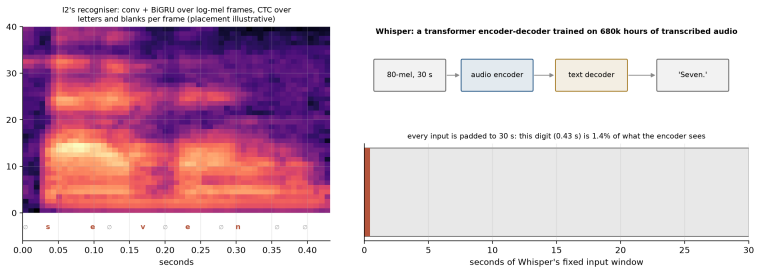


Figure 7: The two recognisers

Left: the log-mel frames of "seven" that the CTC model reads, with an illustrative per-frame labelling (letters where the sound is, blanks elsewhere). Right: Whisper's pipeline, and how little of its fixed 30-second window a spoken digit fills.

The math

CTC does not know what a frame is. It needs a monotonic alignment between T inputs and $U \leq T$ outputs, and marginalises over every alignment that collapses to the target:

$$p(y \mid x) = \sum_{\pi \in \mathcal{B}^{-1}(y)} \prod_{t=1}^{T} p(\pi_t \mid x_t).$$

Nothing in that sum cares whether x_t is a column of pixels or 8 ms of audio, which is why the loss, both decoders and the error metrics came across from H3 with no change beyond an alphabet argument.

What does not transfer is the split. The thing a speech model most easily memorises is a *voice*. Each speaker here records every digit fifty times, so a shuffled split puts forty-nine examples of one person saying "seven" in training and asks about the fiftieth — measuring recall of a voice, not recognition of a word. The rung trains the identical model both ways so the size of that is on the record.

Cepstral mean normalisation. Write a log-mel frame as a sum of a speaker term, a channel term and the phonetic variation over time. The first two are approximately constant across an utterance, so subtracting the utterance's own mean removes them and leaves the third. That is

the argument, and it is old; the rung makes it an arm because I wrote the opposite into the code first.

What we measure, and why

- **CER** (character error rate, H3) and **WER** (word error rate) on a speaker the model never heard. For isolated digits WER is simply the fraction of digits misrecognised.
- **Held-out speaker vs shuffled split** — the same model scored when test clips come from *unseen* speakers versus from speakers also in training. The gap measures how much a random split flatters a speech model by letting it memorise voices.
- **Normalisation arms** — per-utterance vs corpus-wide feature normalisation. It turned out to be the single largest effect, which is why it is an arm and not a footnote.
- **Whisper on matched clips**, with transcripts cleaned the same way for both models (numerals mapped to words, homophones like "to"/"two" handled). Without that, the metric would be scoring spelling conventions rather than recognition.

What I built

A conv-over-frequency + BiGRU CRNN trained with CTC on 800 clips from four speakers, validated on a fifth and tested on a sixth. Greedy and beam decoding from H3. Whisper-tiny on the same test clips, resampled 8→16 kHz. Two normalisation arms, and a shuffled-split control.

Results

Medians over seeds 0–2.

arm	val CER	test CER
global normalisation	0.843	0.660
per-utterance CMN	**0.220**	**0.315**

Unanimous across seeds (`normalization/best_is_utterance` = 1 on all three), worth **0.291 CER** at the median and 0.179–0.500 across the range.

split	CER	
speaker held out	0.315	what the rung reports
clips shuffled	**0.063**	what a careless split would report

5.0×, range 4.3–6.0, with only the split changed.

Against the pretrained baseline, on the same 40 clips:

	CER	WER	exact
whisper-tiny	0.757	0.450	0.550
this rung's CTC model	**0.121**	**0.325**	**0.675**

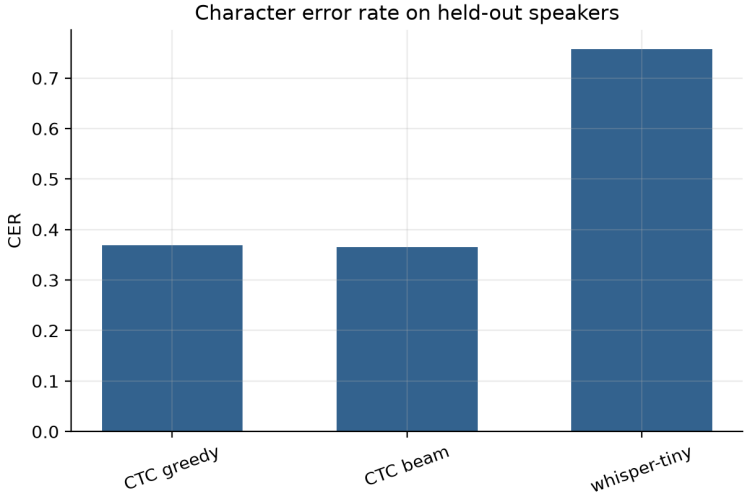


Figure 8: I2 error rates

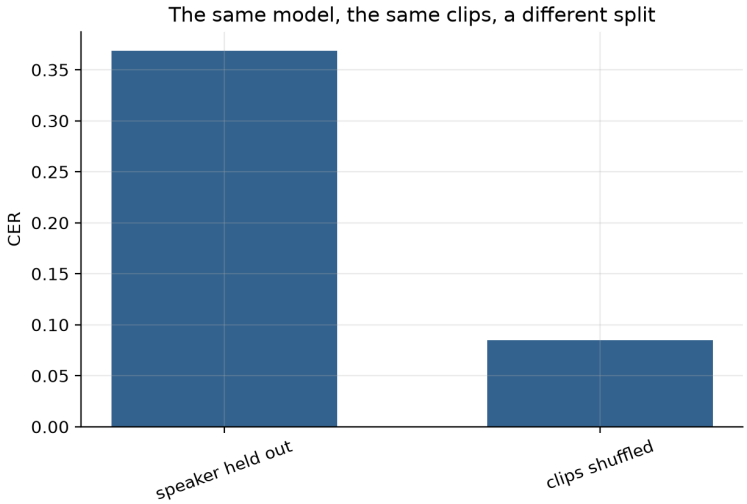


Figure 9: I2 leak

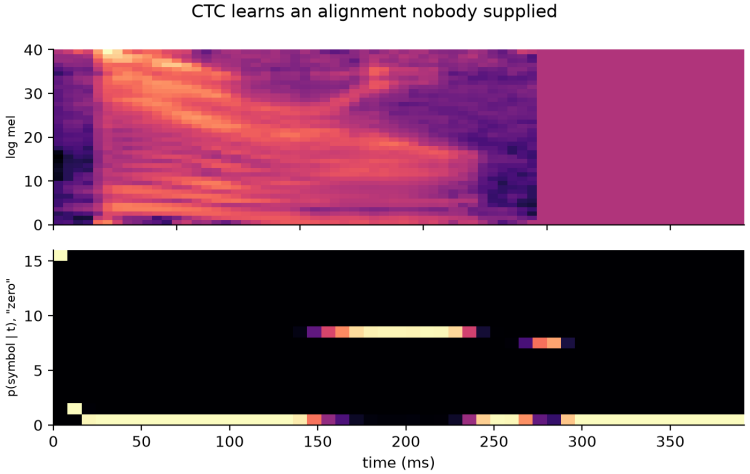


Figure 10: I2 alignment

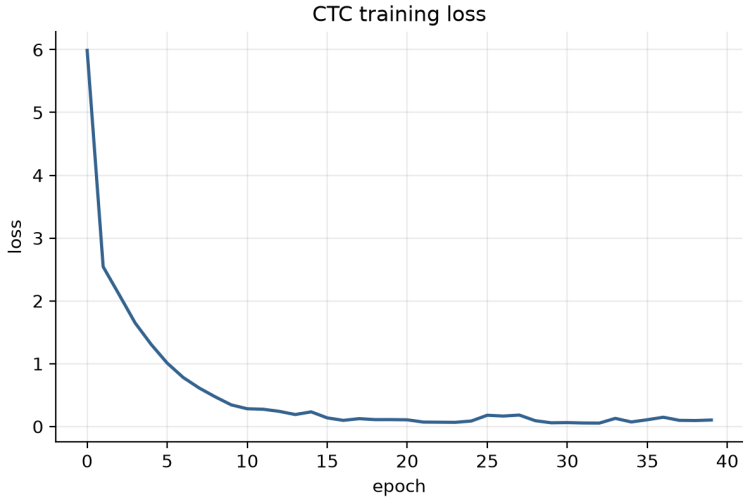


Figure 11: I2 training

What surprised me

I argued myself into the wrong default and the loss curve caught it. The rung normalised by corpus statistics, with this in the docstring: per-utterance normalisation "would quietly remove exactly the speaker differences this rung is trying to measure generalisation across". That is backwards — the rung is not trying to *measure* speaker differences, it is trying to generalise across them, and removing the speaker's average spectrum is the standard way. The symptom was unmissable once I looked: training loss **0.002** with validation CER **0.90**. The model fit four voices perfectly and read a fifth no better than silence. Making it an arm was worth 0.291 CER, more than any architectural choice in the rung.

A 39M-parameter pretrained model lost to a 900k-parameter one trained on 800 clips, and the reason is domain rather than capability. Whisper expects 16 kHz wideband conversational speech; it is being handed 8 kHz band-limited isolated digits, upsampled, with no context for its language model to use. Its errors are *whole-word substitutions* — "three" heard as "Please." — which is why its CER (0.757) is far worse than its exact match (0.550) suggests. That is part H4's finding arriving from the other direction: an autoregressive decoder concentrates error into fewer, larger units and CTC scatters it.

A null result I built the apparatus for and will report as one. Expecting "two"→"too", I wrote a homophone fold. It explains **0%** of Whisper's errors on this speaker, on every seed. The hypothesis was reasonable and the measurement says no.

And a scoring bug of my own that mattered elsewhere. Whisper sometimes writes a spoken digit as a numeral — "1.", "3" — and the rung stripped non-alphabetic characters before scoring, turning correct answers into empty strings. In *this* rung the fix changed nothing, because multilingual whisper-tiny spells digits as words. In the candle comparison, which uses `tiny.en`, the same fix moved the score from 0.25 to 0.75. Same bug, one place it mattered and one place it did not, and no way to know which without checking both.

Open threads

Four training speakers is very few for speaker independence, and the gap between 0.063 shuffled and 0.315 held-out is mostly that. SpecAugment and speed perturbation are the cheap next controls.

Further reading

Graves et al. (2006) for CTC; Radford et al. (2022) for Whisper.

I3 — Text-to-speech, evaluated without listening

Question. Synthesis is scored by human listening tests. What can you honestly conclude from a synthesiser you cannot hear?

The idea

Text-to-speech (**TTS**) runs recognition in reverse: from text, produce audio. Modern systems split it in two:

1. an **acoustic model** predicts a mel spectrogram from the text: what the sound should *look* like over time. Here it is tiny: an embedding of the digit and a GRU that emits a fixed 48 frames;
2. a **vocoder** turns that spectrogram into a waveform. The hard part is that a spectrogram records only *how much* energy each frequency has, not its **phase** (where in its cycle each wave is), and a waveform needs both. **Griffin-Lim** guesses the phase and refines the guess iteratively until the result's spectrogram matches the target. Neural vocoders learn this instead.

Synthesis quality is normally judged by people listening. This rung asks what can be concluded honestly *without* a listening test, using objective metrics and the controls that make them readable.

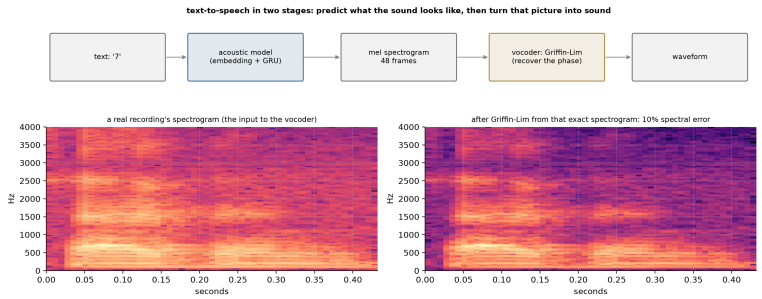


Figure 12: The TTS chain, and the vocoder's own ceiling

Top: the two-stage chain. Bottom: Griffin-Lim given a real recording's exact spectrogram. It still loses about 10% (visible as smearing and noise), so even a perfect acoustic model could not do better than this through this vocoder.

The math

Griffin-Lim is phase retrieval by alternating projection. An STFT magnitude has discarded the phase, and phase is most of what a waveform is. Starting from random phase, project alternately onto "has the right magnitude" and "is the STFT of some real signal":

$$x_{k+1} = \mathrm{ISTFT}\left(|S| \cdot \frac{\mathrm{STFT}(x_k)}{|\,\mathrm{STFT}(x_k)|} \right).$$

Both sets are non-convex, so this converges to a fixed point that need not be the global one. On a *pure tone* — one consistent phase assignment, the easiest possible case — the tests measure it falling from 0.21 error at 16 iterations to 0.05 at 256 and then stopping. It does not get there.

The metrics, and what each is blind to. Mel distance is what the model optimises, so it says whether training worked and nothing about whether the output is good — a blurry average of the training set scores well. An ASR round trip asks the one question with an objective answer, *is the intended word recoverable*, and is blind to everything ASR is invariant to, which is most of what makes speech sound human. A duration and speaker-envelope check catches the failure the first two miss: collapsing every voice into one.

What we measure, and why

- **Mel distance** between synthesised and real spectrograms — how close the acoustic model's picture is. It is read against two controls: a constant "average digit" spectrogram (a model that learned nothing) from above, and the distance between two real recordings of the same digit (natural variation) from below.
- **Intelligibility by machine**: run I2's recogniser on the synthesised audio and measure CER. The chain *clean speech* → *vocoded speech* → *synthesised speech*, all on one voice, separates what the vocoder loses from what the acoustic model loses.
- **Duration error** — how far the length of the synthesised speech (its voiced frames) is from a real recording's, a crude check that catches output that is silent, clipped or too long.
- **Speaker match** — which training speaker the synthesised voice is closest to. A model trained on several voices can average them into a voice that belongs to nobody.

What I built

An embedding-plus-GRU acoustic model mapping a digit to a fixed 48-frame mel, Griffin-Lim as the vocoder, and three metrics each with the controls that make them readable — a constant-spectrogram baseline above and the spread between two real recordings below, plus a clean→vocoded→synthesised chain run on one voice.

Results

Medians over seeds 0–2.

mel distance	
acoustic model	**0.636**
a constant spectrogram (must beat)	1.504
two real recordings of the same digit (noise floor)	1.673
vocoder on real audio	2.761
whole pipeline	3.955

The ASR round trip, one voice through three stages:

	CER	step cost
clean audio	**0.000**	—
through the vocoder	0.050	**+0.025**
synthesised	0.500	**+0.400**

	CER	step cost
(*judge on a held-out human voice*)	0.236	—

Duration error **26 ms** against a 331 ms reference (8.0%). Speaker match: **0 of 3 seeds** — the synthesised voice is consistently closest to a speaker the model never trained on, margin −0.90.

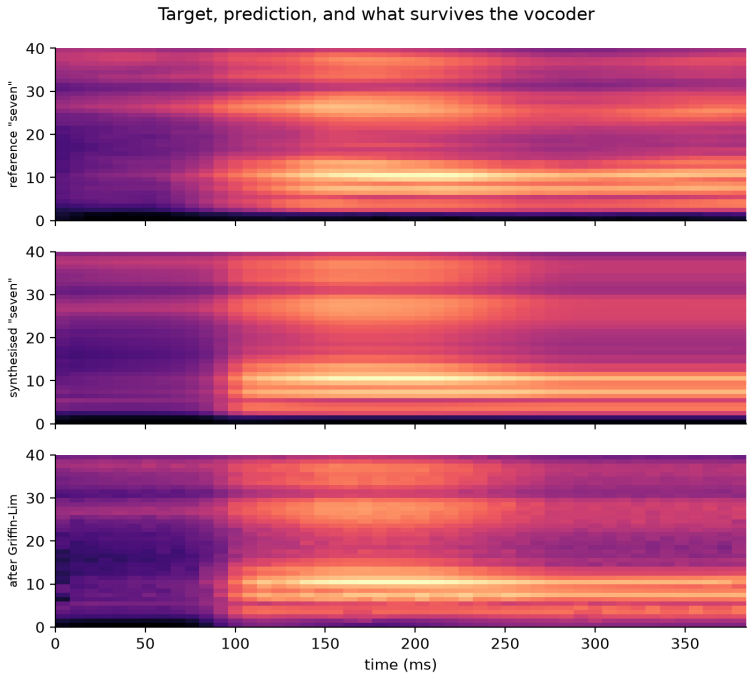


Figure 13: I3 synthesis

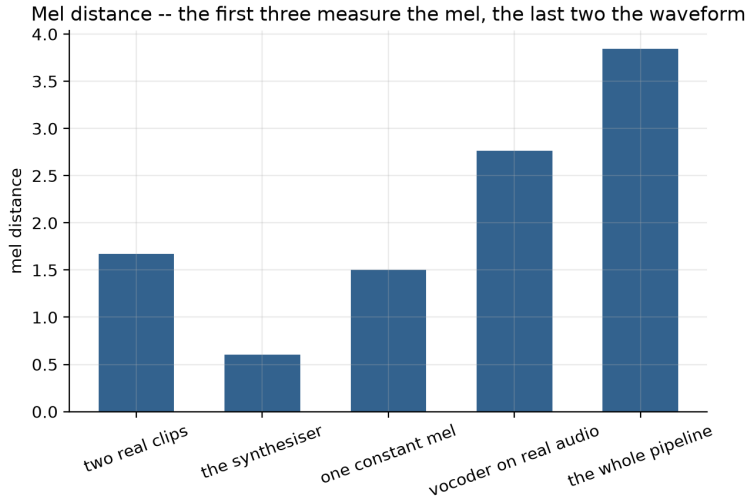


Figure 14: I3 metrics

What surprised me

The vocoder costs everything on one metric and almost nothing on the other. Griffin-Lim costs **2.761 mel distance** — four times the acoustic model's own 0.636 — and **0.025 CER**. It destroys a great deal of spectral detail and almost none of the intelligibility. I had written into the rung that it "puts a ceiling on quality that everything downstream inherits"; that is true of one metric and

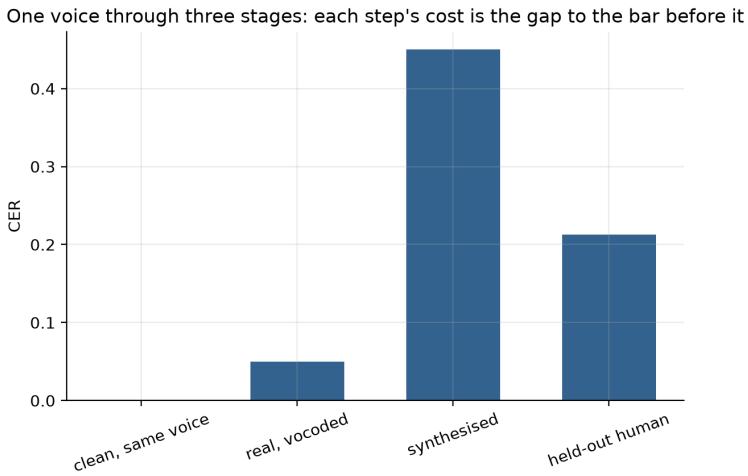


Figure 15: I3 round trip

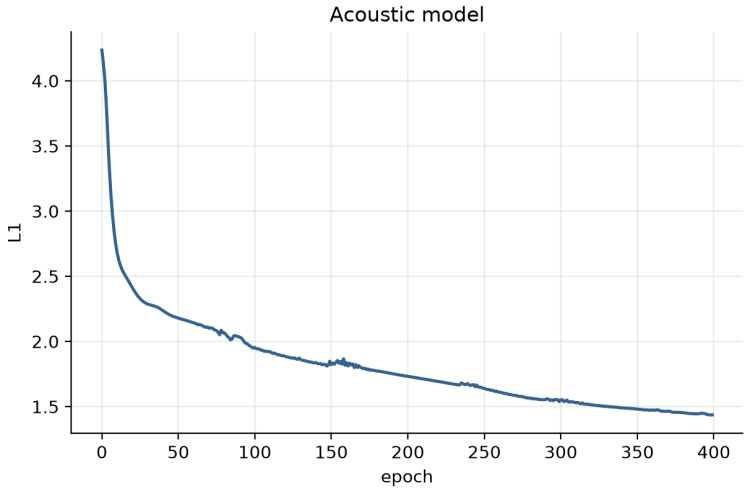


Figure 16: I3 training

nearly false of the other, and 0.400 of the synthesised audio's 0.500 error is the acoustic model's, not the vocoder's.

That number was only trustworthy after the judge was fixed. The first version built the round-trip recogniser with an `Experiment` carrying no normalisation key, so it silently used the global arm I2 had just measured as costing 0.29 — and read *clean human speech* at 0.905 CER. It reported the vocoder's cost as exactly 0.000, which looked like a clean result and was a judge unable to tell anything apart. A round-trip metric is worth exactly what the recogniser behind it is worth, and there is no way to see that from the metric.

I also compared the wrong things twice. The vocoder "ceiling" was quoted against the acoustic model's mel distance, putting a waveform round-trip beside a spectrogram prediction — which is how a synthesiser appeared to score *below* its own floor. And the vocoded control was compared against a held-out speaker's number when the vocoded audio comes from the training voice. Both are now like for like.

The speaker check fails, reproducibly. At 400 epochs on 200 clips from one voice, the synthesised envelope is closer to a speaker the model never saw. Mel distance says the model learned the words; the speaker metric says it did not learn the voice, and only having the third metric makes that visible.

Open threads

A neural vocoder (HiFi-GAN) would separate "Griffin-Lim is lossy" from "the acoustic model is blurry" properly, and duration prediction — fixed at 48 frames here — is the obvious next piece. No listening test was run and nothing here should be read as a claim about naturalness.

Further reading

Griffin & Lim (1984); Shen et al. (2018) for the two-stage split.

I4 — Streaming and latency

Question. Every number in I2 used the whole utterance. What does answering *while* someone speaks cost?

The idea

Every result in I2 used the **whole** recording before answering. A live system (dictation, voice assistants, captions) has to answer **while** the person is still speaking, and that changes the problem.

I2's recogniser is **bidirectional**: its understanding of each moment uses audio from *after* that moment, which does not exist yet in a live stream. There are three standard responses:

- **chunking** — process audio in short blocks as it arrives, each one on its own, so the model never sees past the end of its current block;
- **lookahead** — wait for a little future audio before committing to an output: better accuracy, bought with delay;
- **a causal model** — one built to only ever look backwards, so it needs no future at all.

The cost of each is **latency**: how long after a sound is spoken its transcription can appear.

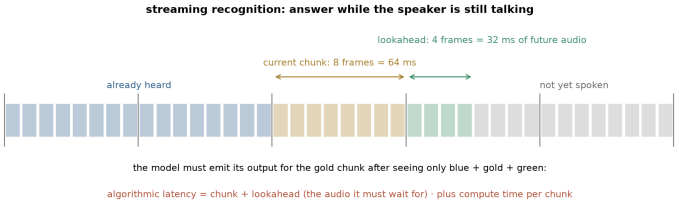


Figure 17: Chunks, lookahead and latency

Audio arrives in 8 ms frames. To emit its output for the current chunk (gold) the model may use everything already heard (blue) plus the lookahead (green), so it must wait for the end of the chunk plus the lookahead.

The math

The whole rung is one inequality. Write $R(t)$ for the set of input frames the output at frame t is allowed to depend on. The three designs differ only in that set:

$$R_{\text{bidir}}(t) = \{1..T\}, \qquad R_{\text{chunk}}(t) = \left\{1..\min\left(T,\ \left\lceil \tfrac{t}{C} \right\rceil C + L\right)\right\}, \qquad R_{\text{causal}}(t) = \{1..t\},$$

and the algorithmic latency is how long you must wait for max $R(t)$ to arrive: $T \cdot H$ for the first, $(C + L)H$ for the second, H for the third, with H the hop. Accuracy and latency are not two quantities to balance — they are the same choice of R, read from two ends.

I2's recogniser is **bidirectional**: the representation of frame t depends on every frame after it. Free at evaluation time, impossible in a stream. Three responses, and the rung measures all three:

1. **Chunk it anyway** — run the bidirectional model on independent windows of C frames. Wrong at every boundary, and the error measures how much future context the model was using.
2. **Buy it back with lookahead** — emit frame t only after L more frames arrive. Latency is L frames; the question is the exchange rate.
3. **Train a causal model** — one that never reads backwards, pays a fixed cost and has no lookahead latency at all.

Algorithmic latency and wall-clock are reported separately, because the first is a property of the architecture and reproduces anywhere while the second is a property of this CPU today.

What we measure, and why

- **CER against chunk size and lookahead** — how much accuracy streaming costs, and how much lookahead buys back.
- **Algorithmic latency** (ms) — how long the model must wait for audio, from the chunk and lookahead alone. It is a property of the design, so it reproduces on any machine.
- **Wall-clock time per chunk** — the compute delay on this CPU, reported separately because it is a property of the hardware on the day, not of the design.
- **The causal model's CER**, with its parameter ratio to the bidirectional one stated (1.22×, not matched), since a comparison between models of different size has to say so.

What I built

I2's recogniser, chunked at 4–32 frames with 0–16 frames of lookahead, against a unidirectional model of the same shape trained the same way.

Results

Medians over seeds 0–2. Full utterance: **0.429 CER**.

design	CER	latency
full utterance, bidirectional	0.429	392 ms
chunked 8 frames, no lookahead	0.721	64 ms
chunked 32 frames, no lookahead	0.572	256 ms
causal model	**0.673**	**8 ms**

Lookahead buys accuracy back at a measurable rate:

lookahead	0	2	4	8	16 frames
CER	0.721	0.672	0.639	0.619	**0.579**

0.148 CER recovered for 128 ms, or 0.0115 CER per 10 ms, with 0.127 still separating it from the full-utterance model. Chunking costs **4.9× the compute** for the same audio, because each window re-reads the utterance from its start.

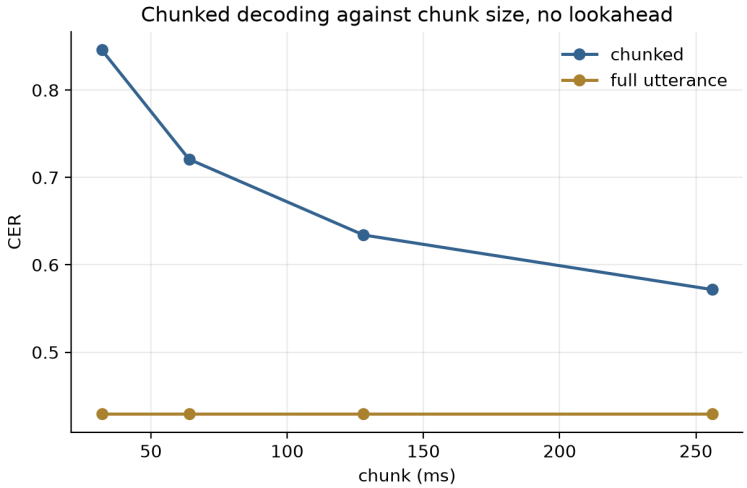


Figure 18: I4 chunk

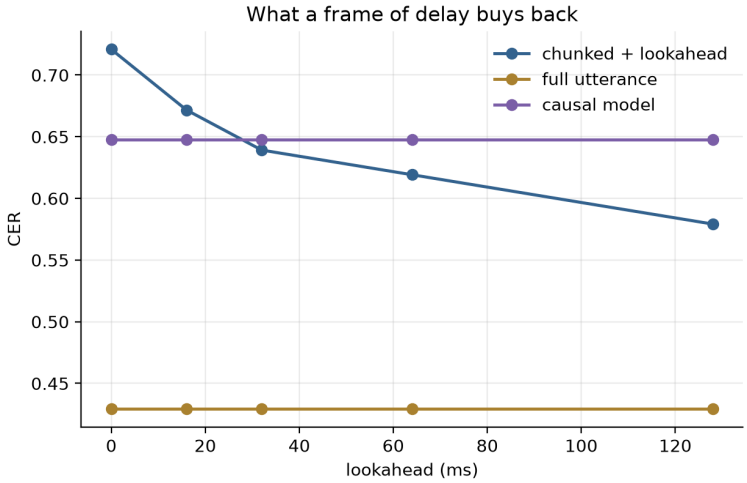


Figure 19: I4 lookahead

What surprised me

Designing for the constraint beat retrofitting it, by more than the parameter count explains. The causal model at 0.673 beats the chunked bidirectional model at 0.721 while having *zero* lookahead latency — it is as good as roughly four frames of delay, for free. The bidirectional model learned to use future context and then had it taken away; the causal one never depended on it.

The causal model is 1.22× the parameters, not matched, which I claimed in the docstring before measuring. Doubling the hidden size of a unidirectional GRU does not equal a bidirectional one of half the width: the recurrent matrix is $(2h)^2$ against two of h^2. The direction is at least conservative — the causal arm is the *larger* one, so the accuracy it gives up is a cost of causality rather than of capacity — but "matched" was wrong and is now reported as a ratio.

Streaming costs compute, not just accuracy. 4.9× for the same audio, because this implementation re-reads from the start on every chunk. A production system carries the recurrent state

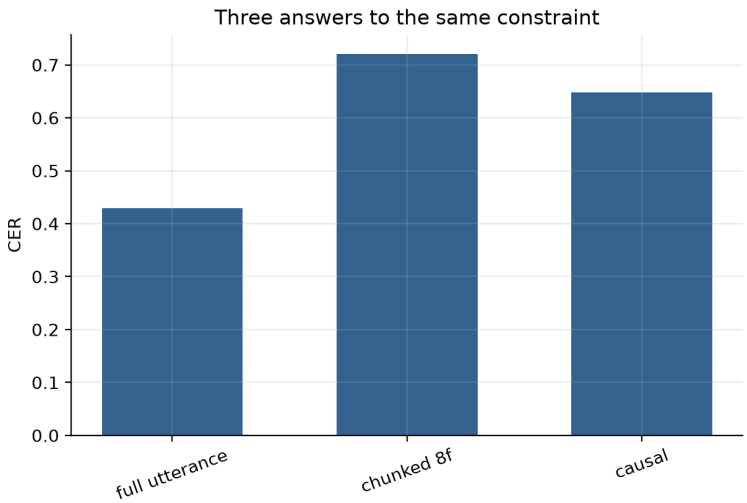


Figure 20: I4 designs

forward instead; that this one does not is the honest limitation, and it is why the wall-clock number is kept separate from the algorithmic latency that does transfer.

Open threads

Truncating left context as well as right would make this a real streaming model rather than an upper bound on one, and carrying the GRU state across chunks would remove the 4.9×. Neither is implemented.

Further reading

the torchaudio streaming tutorials in this part's lab reference.

Whisper through the candle fork, actually run

Part I's own Whisper numbers (I2) come from transformers. The accompanying candle fork implements Whisper too, and ignite-catalog serves it through a `candle-transcription` executor, so both were run on a Kaggle T4 on this part's held-out speaker. Raw records: `data/i-real-models.json` (summary) and `data/i-whisper-executor.json` (every transcript).

Two different pieces of code, and the difference matters. The first run used `candle-examples/examples/whisper`, built from the fork. The fork does not change that example, so it is upstream candle's code. The second ran the catalog's executor itself: its own request path (symphonia decode, a linear resampler, candle's mel), its own `Decoder`, one request per file exactly as the handler does, with only the ignite SDK plumbing removed (request parsing and the asset loader, replaced by a local directory holding the same four files). They share the model weights and the network, and almost nothing that decides the output.

The example: empty transcriptions, and why

candle's whisper *example* returns an **empty transcription for all 40 isolated digits**, while transformers reads the same files at 0.75. A control rules out the harness: the same binary transcribes the example's own long samples (JFK, a news clip) correctly.

The cause, measured. The example ran with timestamps on (its `--timestamps` flag defaults to true and cannot be switched off), and every digit came back as one timestamped span with nothing in it. So the window was decoded and kept, not dropped by the no-speech skip. To find out why, upstream's current example (huggingface/candle@c68b2499; the fork's copy is identical in these lines) was built on CPU and run over this part's clips with each suspected defect toggled separately, printing the token ids.

The ids show what happens. For `tiny.en`, `50358` is `<|transcribe|>`, `50363` is `<|0.00|>` and `50256` is end-of-text:

```
"zero", take 1   [SOT, <|transcribe|>, <|1.78|>, EOT]                -> ""
"zero", take 0     [SOT, <|transcribe|>, <|1.80|>, <|1.80|>, text ...]   -
> empty segment, then a hallucination
```

Two defects compound:

1. **The prompt.** English-only checkpoints were trained on `<|startoftranscript|>` alone, with no language and no task token. The example always appends `<|transcribe|>`, which these models never saw, and with it `tiny.en` opens with a late timestamp, around 1.8 s on clips whose speech starts at 0 s. This is the same out-of-distribution token as the executor's defect 4 below.
2. **The pairing rule.** Whisper's timestamps come in pairs that bracket a segment. The rule that enforces this (openai/whisper's `ApplyTimestampRules`, transformers' `WhisperTimeStampLogitsProcessor`) checks whether the previous token was also a timestamp, and when fewer than two tokens have been sampled it answers *yes* (`len(seq) < 2 or ...`). So after the initial timestamp the next token must be text. candle's port answers *no*. It therefore reads the initial timestamp as an open pair, masks out every text token, and leaves the model only two moves: close an empty segment or stop.

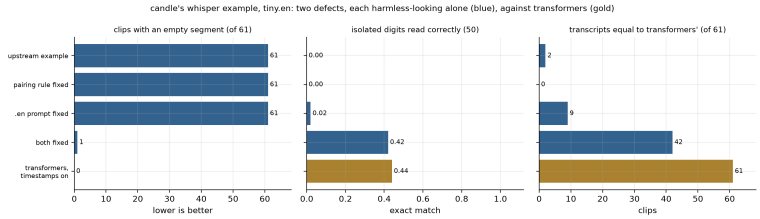


Figure 21: Empty transcriptions: each fix toggled

`tiny.en`, CPU	digits exact	sequence WER	JFK	clips with an empty segment	words equal to transformers'
upstream example	0.00	0.925	perfect	61/61	2/61
pairing rule fixed only	0.00	1.000	**empty**	61/61	0/61
prompt fixed only	0.02	0.375	perfect	61/61	9/61
both fixed	**0.42**	**0.200**	perfect	**1/61**	**42/61**
both, temperature fallback off	0.44	0.200	perfect	0/61	44/61

`tiny.en`, CPU	digits exact	sequence WER	JFK	clips with an empty segment	words equal to transformers'
transformers, timestamps on	0.44	0.150	perfect	0/61	—

Ten four-digit sequences, the 50 digit clips (resampled to 16 kHz, which the example requires), JFK. "Empty segment" counts a timestamp pair with no text between, or an opening timestamp followed by end-of-text, read off the token ids.

Neither fix works alone, which is why this was hard to see. With only the pairing rule fixed, the model still opens late, and now that text is required after the first timestamp it ends the transcript instead, so even JFK comes back empty. With only the prompt fixed, every clip still has an empty segment. Together they come within one digit clip and two sequence words of transformers' scores. The reference's default cap on the first timestamp (1 s, which the example leaves unset) changes nothing once the prompt is right, because the first timestamp is then 0.00 s, as in transformers. The multilingual `tiny` improves too: digits 0.12 → 0.26, and empty segments 59 → 0 of 61. A smaller third bug: printed span times run 40 ms late, because `<|0.00|>` is the token *after* `<|notimestamps|>`, not two after.

The fix is 17 lines and is on the fork as davidmiheev/candle#13, ready to send upstream; the patch is also in `tools/real_models/candle-patches/`. Raw token ids and transcripts for every arm: `data/i-whisper-example-rules.json`, made with `tools/real_models/whisper_example_rules.py`.

Building the example at all required two fixes at the time. `candle-transformers` in the fork did not compile without CUDA, contradicting the catalog's own `whisper-cpu.yaml`, which declares `gpu.required: false`. The fork has since added a CPU stub for the missing module. Also, `candle-examples` declared symphonia 0.6 while its audio code used the 0.5 API. That declaration is upstream's (a dependency bump, huggingface/candle#3546), and upstream has since ported the audio code to 0.6 (huggingface/candle#3945), so the fork's 0.5.4 pin is no longer needed once it syncs.

The executor: it transcribes, and it has four defects of its own

Seven arms, 61 requests each: 50 isolated digits (the raw 8 kHz files, which the executor resamples itself), ten four-digit sequences, and the JFK clip. Two models: `tiny.en`, and `large-v3-turbo`, the model the catalog registers for this executor. transformers ran on the same bytes, both with its own `generate` and with a greedy loop that copies the executor's decode step by step. The loop reproduces `generate` exactly (61/61 on tiny.en), so it is a faithful stand-in.

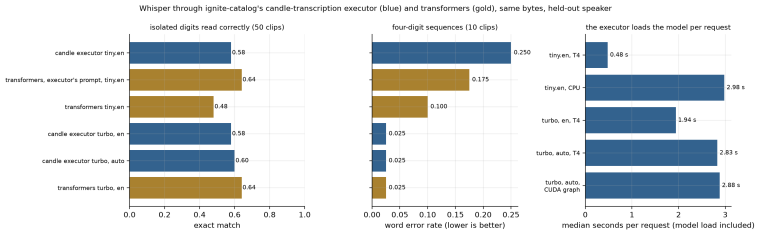


Figure 22: The executor against transformers

	digits exact	sequence WER	JFK
executor, tiny.en (eager = static = CUDA graph = CPU)	0.58	0.250	perfect
executor, large-v3-turbo, language auto	0.60	0.025	perfect
executor, large-v3-turbo, language=en	0.58	0.025	perfect
transformers, tiny.en	0.48	0.100	perfect
transformers, large-v3-turbo	0.64	0.025	perfect

What this establishes:

1. **The executor never returns an empty transcription**: 0 of 427 requests across all arms. The example's failure does not reach the production decode path.
2. **The fork's optimised paths are exact.** Static decode, the captured CUDA graph and CPU all produce **byte-identical text** to eager decoding on every request, for both models. (They are not faster here, because the executor reloads the model on every request and that dominates: 0.48 s per tiny.en request on the T4 against 2.98 s on CPU; 1.9–2.9 s for turbo.)
3. **Language detection is meaningless on English-only models.** The executor picks the highest-scoring `<|xx|>` token from the vocabulary, and `tiny.en` has those tokens but was never trained on them. It reports JFK as **`sq`** (**Albanian**) and the digits as `ms`, `km`, `hi`, `ca`, and the reported language even changes between eager, static and CPU runs whose text is identical. The language is reported and not injected, so the text is unaffected, but the response's `language` field is wrong for every `.en` model.
4. **The prompt carries a token English-only models never saw.** The executor always decodes from `[SOT, (language), transcribe, notimestamps]`, while English-only Whisper was trained from `[SOT, notimestamps]`. Feeding transformers the executor's prompt reproduces the effect: isolated digits go from 0.48 to **0.64** and sequences from 0.10 to **0.175** WER. The out-of-distribution token changes behaviour, better on single words and worse on runs of them, and matching it raises candle/transformers agreement from 32 to **46 of 61** transcripts.
5. **Token suppression is silently off for the registered model.** The executor reads `suppress_tokens` from `config.json`. `large-v3-turbo` keeps them in `generation_config.json`, so the executor suppresses nothing where transformers suppresses the usual list.
6. **The asset name says 80 mel filters, and 80 would crash.** The executor loads `melfilters80.bytes`. `large-v3-turbo` uses 128 mel bins, and handed a genuine 80-filter file the request **panics** (`index out of bounds` in candle's `whisper/audio.rs`). So production either stores 128 filters under the name "80", or this path crashes on the model it serves. Which one is not visible from outside the asset store.

Also measured: auto language detection runs the encoder twice (once to detect, once to decode), which costs **0.9 s per turbo request** (2.83 s against 1.94 s with the language given), and omitting the language token changes 13 of 61 turbo transcripts (`Oui.` against `We.` for a spoken "three", `3.` against `three.`).

The executor's defects, fixed and re-measured

The four defects are fixed on ignite-catalog's `fix/candle-transcription-executor` branch: - the English-only prompt is `[SOT, notimestamps]`, and language detection is skipped for `.en` models; - `suppress_tokens` and `begin_suppress_tokens` come from `generation_config.json`; - the mel filters are chosen by `num_mel_bins` and checked before the weights load; - the first window's encoder output is reused instead of computed twice; - the fallback re-decodes a window only when its output repeats (compression ratio above 2.4), never just for low confidence.

Run again on CPU over the same 61 clips:

`tiny.en` executor	digits exact	sequence WER	JFK	words equal to transformers'	reported language
before	0.58	0.250	perfect	37/61	`ms`, `km`, `hi`, `ca`, …
after	**0.48**	**0.225**	perfect	**41/61**	`en`, 61/61
transformers	0.48	0.100	perfect	—	—

Digit accuracy *falls*, and that is the fix working: 0.48 is exactly what transformers scores with the prompt `tiny.en` was trained on, and the 0.58 came from the out-of-distribution `<|transcribe|>` token, which defect 4 showed helps single words and hurts sequences. One window in 61 needed the repetition fallback. Raw transcripts: the `tiny.en/cpu-after-fixes` arm of `data/i-whisper-executor.json`.

What is not explained. With prompts matched, candle and transformers still disagree on 15 of 61 tiny.en transcripts, including hallucinations only candle produces for a single spoken digit ("Thank you for watching.", "for the next couple of months."). The candidates are the mel front end (candle's own FFT, padding and filter file, against transformers' feature extractor) and f32 against fp16. That comparison was not run. One clue: two candle decoders that share only the model and `pcm_to_mel` (not the resampler, not the decoding loop) land at the same distance from transformers: this executor at 41 of 61 transcripts, and the fixed example, with its timestamp rules, at 42 of 61 (44 greedy). That points at what they share.

Open threads

The executor's defects are fixed and re-measured, and the example's empty transcriptions are explained and patched; both are above. Still open: diff candle's `pcm_to_mel` against transformers' feature extractor on the same waveform, stage by stage as I1 does against torchaudio, to locate the remaining disagreement that both candle decoders share. Sending the example's fix upstream to huggingface/candle. And the executor's turbo path, re-measured here only through the `tiny.en` model it shares its code with.

What part I settled

1. **Three stages agreeing to 10^{-6} is what makes the fourth's 3.5×10^{-2} readable.** The MFCC disagreement was a log guard — offset versus clamp — and the quietest mel energy in the corpus sits *below* torchaudio's offset, so it is not guarding, it is shifting. Worth 27,490× the agreement, and invisible to an end-to-end diff.

2. **The time–frequency trade is an identity, not a tendency.** 1920 Hz·ms at every window length from 4 ms to 128 ms, constant to 1.00×.
3. **A shuffled split is worth 5× on this corpus.** 0.063 CER against 0.315, same model, same clips, only the split changed. In speech the thing a model memorises is a voice.
4. **I argued my way into the wrong normalisation and a loss curve caught it.** Training loss 0.002 with validation CER 0.90 is a model that fit four voices and read a fifth as noise. Per-utterance CMN was worth 0.291 CER — more than any architectural choice here — and I had written a paragraph explaining why it would be wrong.
5. **A metric can cost everything on one axis and nothing on another.** Griffin-Lim: 2.761 mel distance, 0.025 CER. Any single-number summary of this vocoder is a choice of which fact to report.
6. **A round-trip metric is worth exactly what its judge is worth, and the metric cannot tell you that.** The broken judge read clean human speech at 0.905 and reported the vocoder's cost as a tidy 0.000. The fixed one reads it at 0.000 and reports 0.025.
7. **Designing for a constraint beat retrofitting it.** A causal model with zero lookahead latency (0.673) beats chunking a bidirectional one (0.721), which is worth about four frames of delay for free.
8. **Running the real models found defects no config could show.** candle's whisper *example* (upstream code, unchanged in the fork) returns empty text for every isolated digit, because of two defects that only fail together: an English-only prompt carrying `<|transcribe|>`, and a timestamp-pairing rule inverted for the first token. Fixing both takes digits from 0.00 to 0.42 against transformers' 0.44 (patch on the fork as davidmiheev/candle#13). The catalog's executor does not (0 of 427 requests empty), and its static-decode and CUDA-graph paths match eager decoding byte for byte. It has four defects of its own: a language report that calls JFK Albanian on English-only models, a task token those models were never trained with (in transformers it raises digit accuracy by 0.16 and worsens sequence WER by 0.075), token suppression skipped for the registered model, and an asset named for 80 mel filters that panics if it holds 80. Details in Whisper through the candle fork.
9. **Five of my own mistakes in this part were metrics, not models.** A scoring rule that counted `"1."` as silence; a judge that could not hear; a ceiling compared against the wrong quantity; a control drawn from the wrong speaker; a parameter count claimed as matched without being checked. The arm-level measurements were right every time. That is the tenth such occasion in this repo, and the only method that has ever caught them is adding the control that could embarrass the result.

Cost of the part: ~2,400 lines including tests, 19 test cases, and a three-seed sweep in about 72 minutes on four CPU cores (I1 2 s, I2 ~1,280 s, I3 ~416 s, I4 ~1,107 s per seed), plus a Kaggle T4 kernel for the candle comparison. I2 dominates because it trains three models per seed: both normalisation arms and the shuffled-split control.

Figures regenerate with `uv run python tools/writeup_figures.py I`; the numbers they are quoted against live in `data/i-metrics.json`. The part README is the short version of this document.